Calloway OS

Omni-Core Telemetry Station

Project Report

Author	Logan Calloway
Platform	XIAO ESP32-C6
Version	v2.0
Date	July 2026

"A centralized, glanceable engineering dashboard designed to monitor a workspace and provide real-time environmental and network data from a single, always-on embedded device – now paired with a live web dashboard for historical trends."

Abstract

This report documents the design, development, and architecture of the Omni-Core Telemetry Station, a custom desktop Internet-of-Things (IoT) dashboard built on the XIAO ESP32-C6 microcontroller running Calloway OS.

The goal of this project was straightforward: build a device that never crashes, always knows where it is, and fills a screen with real, useful data pulled from both the internet and local sensors. Most connected devices work until they don't. A dropped WiFi connection, a bad response from an online service, or a web request that never comes back can quietly bring everything down. This project was designed from the ground up to handle those situations gracefully and recover on its own.

The device pulls the current time from internet time servers, weather from OpenWeatherMap, and reads live environmental data from a TSL2591 light sensor, an SCD40 sensor for CO2, temperature, and humidity, and an SEN54 sensor for particulate matter and VOCs (airborne chemicals given off by things like paint, cleaning products, and new furniture). Location and timezone are detected automatically from the network, which means the device works correctly whether it is sitting on a desk at home or plugged in somewhere else entirely. Everything updates itself with no configuration needed after first setup.

Version 2.0 adds a second way to look at the same data: a live web dashboard, served directly from the device itself at `omnicore.local`, with a full 24-hour history of every reading and a set of charts anyone can pull up from a phone or laptop on the same network. The dashboard and the physical screen are both driven by the same underlying sensor readings, so neither one is ever the "real" source of truth over the other; they are just two windows into the same data.

The software is organized so that fetching data, drawing the screen, and coordinating everything else are kept as separate, independent jobs. This makes the codebase easy to extend as new sensors arrive and keeps individual pieces of code small and easy to understand.

Keywords: ESP32-C6, IoT, embedded systems, telemetry, NTP, OpenWeatherMap, ST7796, capacitive touch, TSL2591, SCD40, SEN54, particulate matter, air quality, C++, Arduino framework

Contents

Introduction

Most connected devices are built to work in one place, on one network, with one fixed setup. They tend to be fragile in ways that only show up after you have been using them for a while. A router change breaks the WiFi, a timezone shift breaks the clock, or a brief power outage leaves the device stuck in a state it can never recover from on its own.

The Omni-Core Telemetry Station was built to be different. The core question driving every design decision was: *what happens when something goes wrong, and how does the device get itself back to a good state without any help?*

That question shaped everything. WiFi credentials are stored in memory that survives a power loss, and the device reconnects automatically if the signal drops. Location and timezone are detected from the network rather than fixed in the code, so moving the device somewhere else does not break anything. A safety timer resets the chip if the main program ever gets stuck. Requests to online services have time limits, so a slow or unresponsive server cannot freeze the display. If WiFi never comes back after six hours, the device restarts itself cleanly rather than sitting broken.

The display side was designed with the same thinking. Every part of the screen that updates often is fully drawn in memory first and then shown all at once, so nothing ever flickers or appears half-drawn. Only the parts of the display that actually changed get redrawn on each cycle. The colors used on screen are defined in one place, and everything, including the settings menu, brightness slider, and reset confirmation, responds to a Night Mode toggle that shifts the whole interface to a red color scheme, which protects the user's night vision and sleep cycle during late-night use.

Version 2.0 extends the project in a direction the original design did not originally plan for: history. The physical screen has always been, by design, a snapshot of right now. It has never tried to show a trend, because a small screen simply does not have room for one alongside everything else on it. The web dashboard exists to answer the question the screen never could: what has this room actually been doing for the last hour, the last six hours, the last day?

This report covers the hardware design, software architecture, key engineering decisions, and the development history of the project from initial prototype to the current version. The full source code is available at `https://github.com/logancalloway/calloway-os`.

Project Goals

The Omni-Core was built around a few core requirements that stayed constant throughout development.

- **Reliability above everything.** The device should run unattended for months without intervention. Every failure mode should have a recovery path.
- **Location independence.** The device should work correctly anywhere with an internet connection. No manual reconfiguration required.

- **Real data only.** Nothing on the display should show a placeholder or a zero when real data is not available yet. The display waits for valid data before drawing anything.
- **Glanceability.** The physical dashboard should communicate everything the user needs at a glance, including time, date, weather, local environmental readings, air quality status, and WiFi status, without requiring any interaction.
- **Expandability.** The architecture should make it easy to add new sensors without restructuring existing code.
- **Trend visibility.** Beyond a single glanceable moment, the device should be able to answer "what has this room been like recently," without needing a separate computer, database, or cloud service to do it.

Hardware Design

Platform

The XIAO ESP32-C6 was chosen as the platform for several reasons. It runs at 160MHz (its processing speed) with 512KB of working memory (RAM) and 4MB of storage (flash), far more than this project currently needs, leaving room to grow. WiFi and Bluetooth are built directly into the chip, so no extra hardware is needed to get the device online. It is also built around a RISC-V processor core, a newer, increasingly common chip design in professional embedded (hardware-level) engineering, making this a good chip to build real-world skills on.

Display

The dashboard moved from an earlier 2.8-inch resistive (pressure-sensing) display to a larger 3.5-inch ST7796 screen with better viewing angles and color, running at 480x320 pixels. It is controlled over SPI, a simple wired connection for talking to nearby chips, through the Arduino_GFX graphics library. The larger screen gave enough room to add a full particulate matter row and an overall air quality status line without crowding the clock, date, or weather. Six separate regions of the screen, the clock, date, local sensor readout, particulate row, air quality row, and status row, are each drawn completely in memory first, then sent to the screen in one clean update, so nothing ever appears half-drawn.

Touch Input

The original physical button was replaced with an FT6236 capacitive touch controller, the same touchscreen technology used in phone screens, wired into the same shared connection (I2C, a simple standard that lets several chips share one pair of wires) as the environmental sensors. Tapping the dashboard opens a full settings menu with scrollable items, a live-dragging brightness slider, and a two-step confirmation screen before a WiFi reset is allowed to happen. A touch's position is only trusted at the moment a finger first touches the glass, since the chip reports an unreliable position at the moment a finger lifts back off.

One quirk worth documenting here: the touch chip's own startup check reports failure on this specific board, even when touch is actually working correctly. Rather than trust that check, the device's software just logs that startup was attempted and treats an actual touch on the glass as the real proof it is working.

Sensors

Sensor	Purpose	Status
TSL2591	Ambient light (lux)	Active
SCD40	CO2, temperature, humidity	Active
SEN54	Particulate matter (PM1/2.5/4/10), VOC index	Active

All three sensors share one wired connection (I2C) on pins D4 (data, SDA) and D5 (clock, SCL), running at 400kHz, so they can all talk to the microcontroller over the same two wires. The TSL2591 can measure light levels from near-total darkness up to bright daylight (up to 88,000 lux, a unit of brightness), and its reading drives both the vertical light-level bar on the dashboard and the Auto Dim brightness feature. The SCD40 measures local temperature, humidity, and true CO2 concentration using infrared light, a proven, accurate way of sensing CO2, updating the top-left of the dashboard. The SEN54 replaced an earlier air quality sensor and adds real particulate matter readings across four size classes plus a VOC index, discussed in detail in Section 4.

Backlight Control

The screen's backlight brightness is controlled with PWM, a technique that rapidly switches the power on and off to act like a dimmer. The Brightness menu item cycles through four fixed steps (25%, 50%, 75%, 100%), and the Auto Dim setting maps the TSL2591's live light reading directly onto that brightness control, so the screen dims and brightens automatically as the room's lighting changes.

PCB Design

The board was designed in EasyEDA Standard (circuit design software) and fabricated by JLCPCB, a circuit board manufacturer. Assembly and soldering were done by hand.

The circuit board (PCB) was designed to be small and practical rather than densely packed, prioritizing clean signal quality and keeping heat away from sensitive components over saving space. The XIAO ESP32-C6 mounts directly to the board, with the USB-C port left accessible from the side of the finished enclosure for reprogramming and power without needing to open the case.

The area under and around the SCD40 CO2 sensor is left as an open gap in the copper rather than filled in solid. CO2 sensing this way depends on accurate local temperature, and a solid sheet of copper right under the sensor would let heat from the microcontroller and power regulator conduct into the SCD40 and throw off its readings. The gap breaks that

heat path and gives the sensor better airflow to read the room's actual air rather than the board's own heat.

Separate plug-in connectors are used for the SEN54 sensor and the display, rather than soldering either directly to the main board, so either one can be serviced or swapped without touching the rest of the board. A separate wire is broken out for backlight control, driven directly from the microcontroller as described in Section 3.5.

Small capacitors, components that smooth out electrical noise on the power lines, are placed throughout the board: larger 10µF capacitors on the main power lines (one each for the 3.3V and 5V supplies), medium 1µF capacitors at the SCD40 and TSL2591, a larger 10µF capacitor at the SEN54 to handle its internal fan's power draw, and small 100nF capacitors placed as close as possible to each component's power pin, to keep electrical noise from leaking into the shared sensor connection. All three sensors and the touch controller talk over that one shared connection, so keeping it electrically quiet mattered more here than it would with only one sensor on the board. The two wires of that connection also have pull-up resistors, small resistors tying each wire to power so it reliably reads as "on" when nothing is actively pulling it "off," sized at 4.7kΩ, a standard value given this connection's speed and how far its wires run. The complete pin assignments and address list are collected in Section 11.

Two revisions of this board have been built. The current revision adds an updated footprint (component pad layout) and 5V power supply for the SEN54, corrected after the first prototype run.

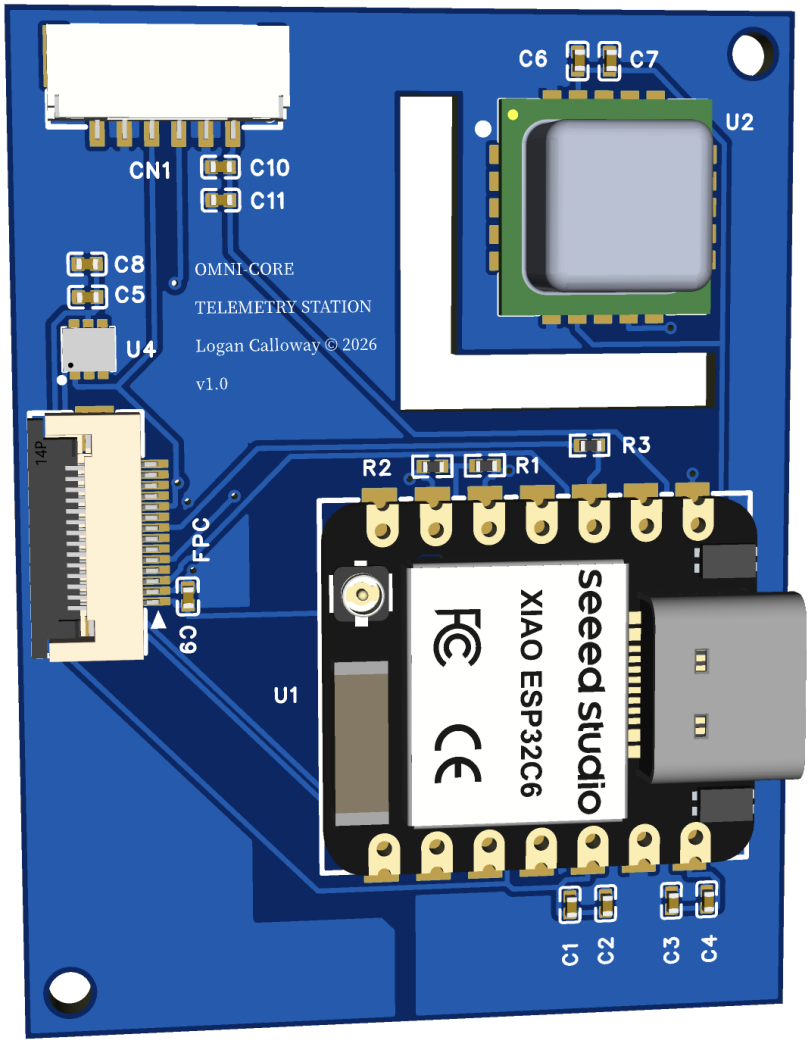

Figure 1: PCB, 3D render, front side.

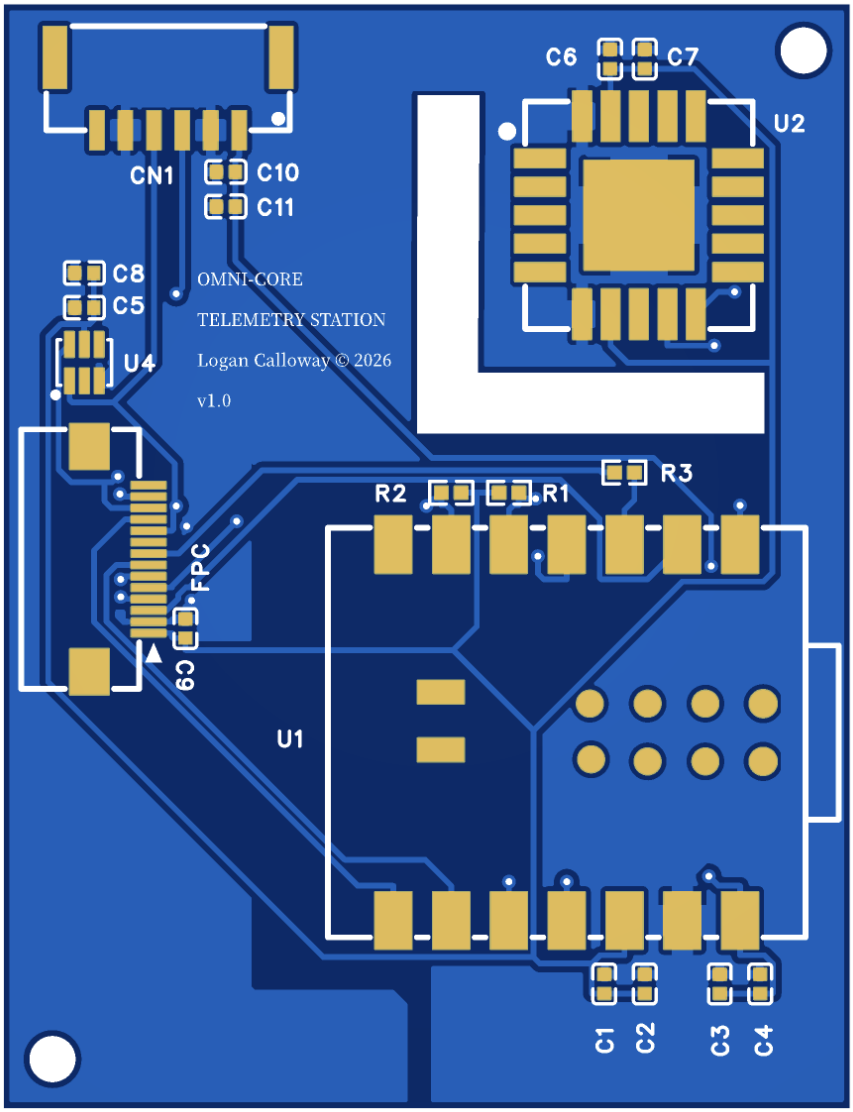

Figure 2: PCB layout, top copper layer.

Figure 3: PCB layout, bottom copper layer.

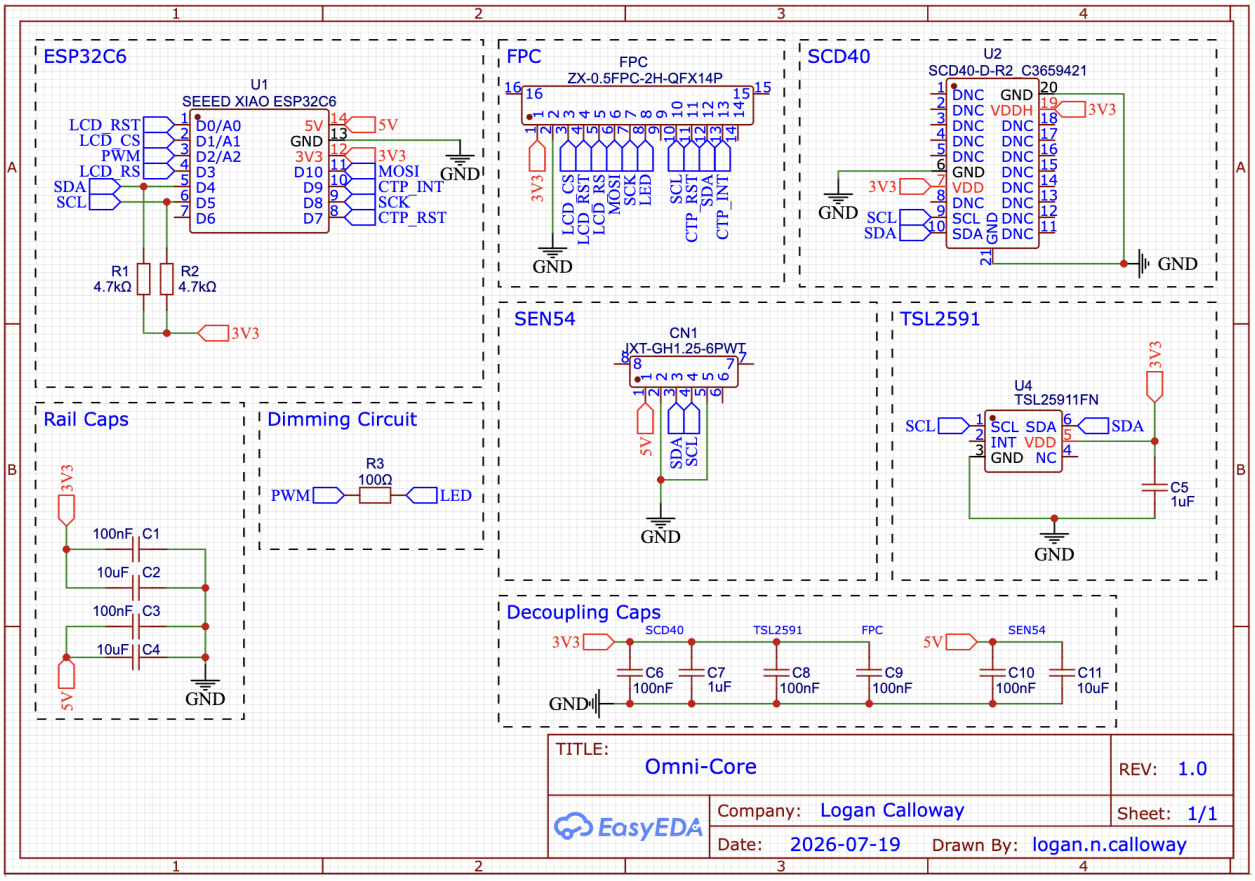

Figure 4: Omni-Core schematic, rev 1.0.

3D Enclosure Design

The enclosure was modeled in SolidWorks (3D design software) and 3D printed.

The enclosure's design constraints came directly from what the electronics actually need physically: airflow to the outside world for the environmental sensors, secure mounting for the PCB and display, and access to the board's connectors without needing screws or tools to open the case.

The back panel uses a hexagonal perforation pattern, open straight through to the outside air. The SCD40's CO_2 reading and the SEN54's particulate reading are only meaningful if the sensors are actually sampling the room's air rather than a sealed pocket of trapped, self-heated air inside the case, so this pattern exists specifically to keep outside air moving through the enclosure. The SEN54 also draws air through its own internal fan, and a separate side vent was added specifically to give that fan a clear, unobstructed path to pull air from, rather than relying on the back panel's general airflow alone.

The PCB mounts inside the main body of the enclosure with the USB-C port aligned to a side cutout, so the board can be powered and reprogrammed without opening the case. A

snugly-sized slot on the inside of the enclosure holds the SEN54 firmly in place against the side vent by friction alone, without needing glue or extra fasteners.

The lid is a separate piece, shaped to fit the 3.5” display exactly and sit flush with the outside face of the enclosure once assembled. Rather than using screws, the lid and body click together with small interlocking tabs molded into both halves, holding firmly under normal handling with no visible screw holes anywhere on the finished enclosure.

A small gap was intentionally left along the left edge of the lid for the display module’s built-in ribbon cable, since the LCD’s own designers routed that cable off the left side of the panel rather than the back.

Figure 5: Enclosure, assembled, front view. The on-screen UI shown is an AI-generated (ChatGPT) mockup of the dashboard, not a capture from the running device.

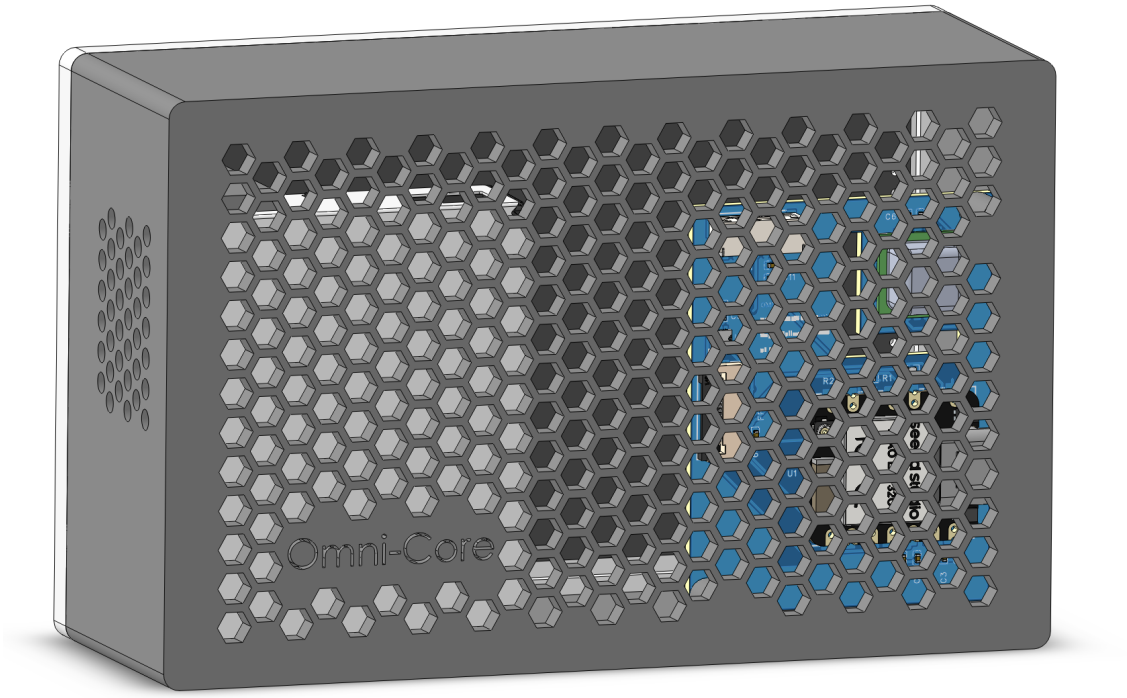

Figure 6: Back panel, hexagonal airflow pattern and side vent.

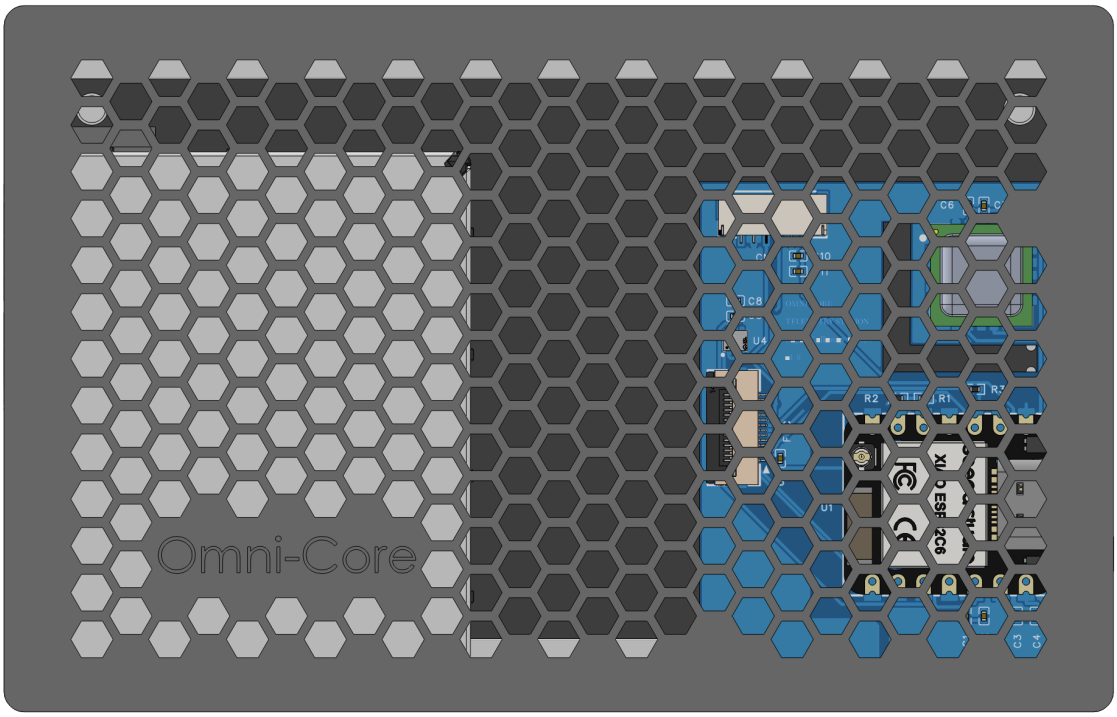

Figure 7: Back panel, straight-on view.

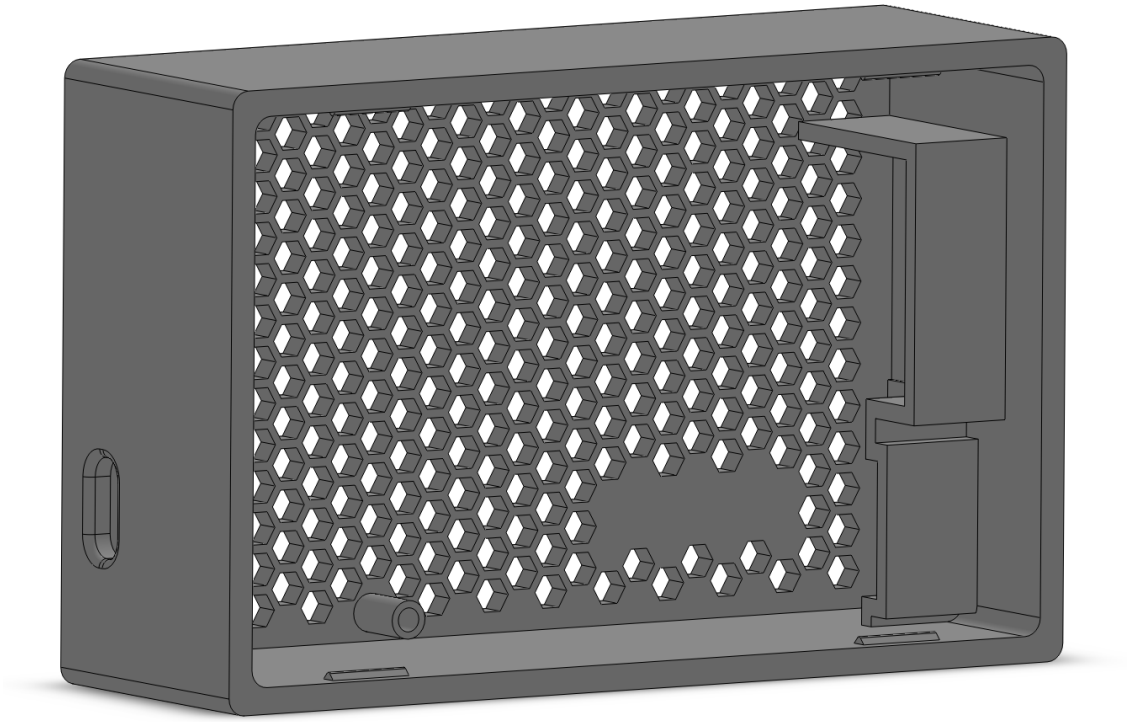

Figure 8: Enclosure body (lid removed), showing the SEN54 press-fit slot against the side vent.

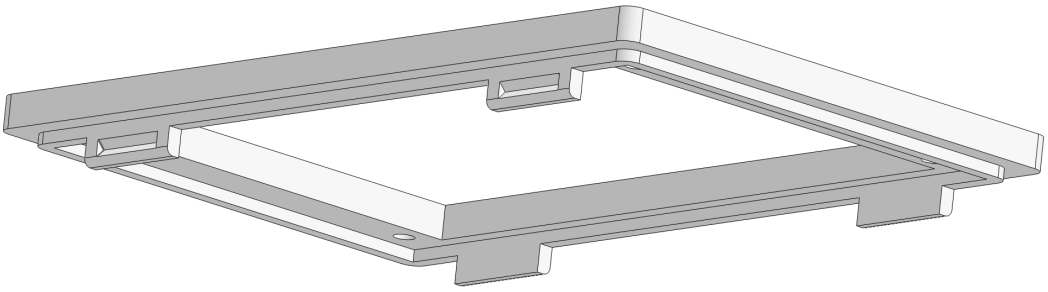

Figure 9: Lid piece, showing the display cutout and snap-fit tabs.

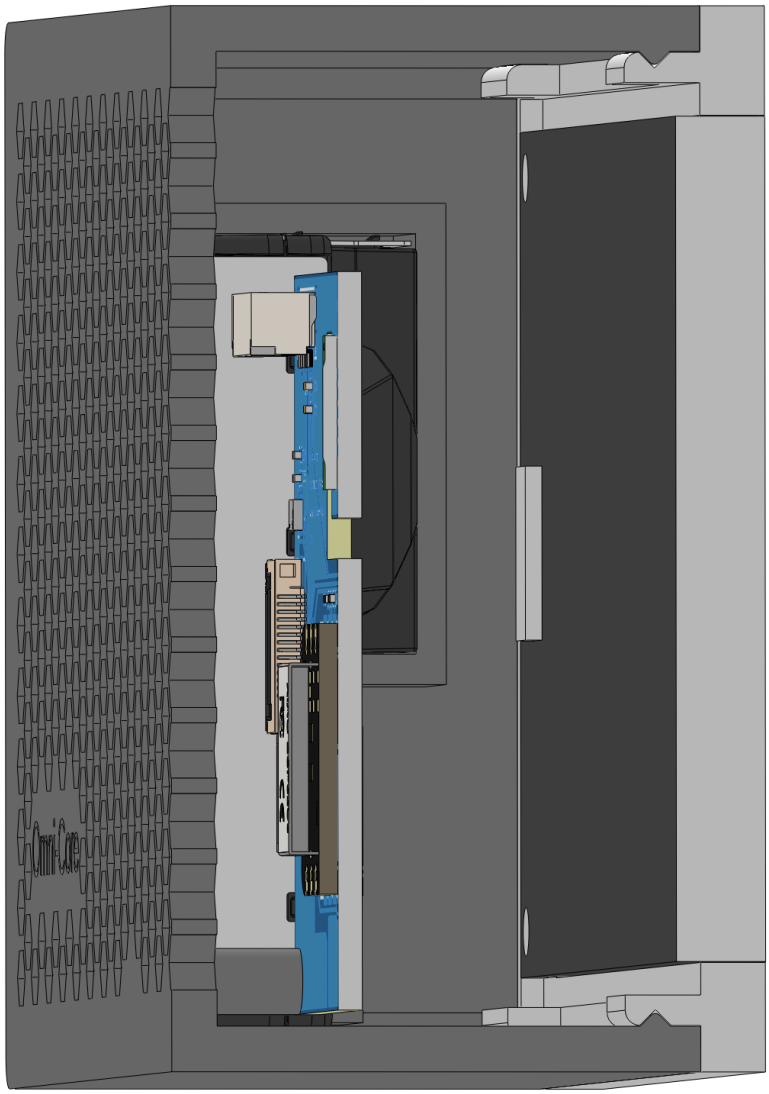

Figure 10: Cross-section through the lid/body joint, showing the snap-fit mating features.

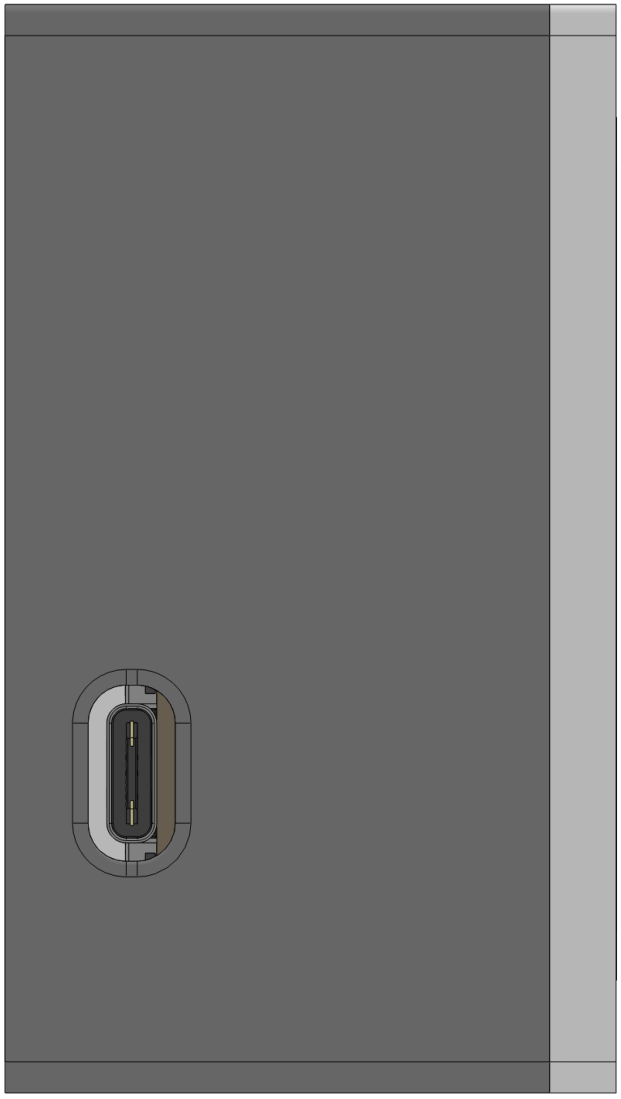

Figure 11: Side view, USB-C port cutout.

Understanding the Air Quality Data

The SCD40 and SEN54 report several different numbers, and it is easy to look at them without actually knowing what any of them mean or why the device bothers to color-code some of them. This section exists so that anyone reading the dashboard, not just someone who already knows air quality science, understands what they are looking at.

CO2 – Carbon Dioxide

CO2 is measured in parts per million (ppm). Outdoor air sits around 420 ppm. Indoors, in a room with people breathing and the windows closed, CO2 climbs steadily, since every exhale adds more of it to the air and normal air exchange with the outside is often too slow to keep up.

CO2 is not just a proxy for stuffiness. It has a direct, well documented effect on cognition. Concentrations above roughly 800 ppm are associated with measurably reduced decision-making performance, and above 1200 ppm the effect becomes pronounced. This is why CO2 is one of the two readings driving the dashboard's overall air quality status: it is one of the few environmental numbers with a real, causal link to how sharp a person feels in a room. The fix is almost always simple. Crack a window or door and the number falls quickly.

Particulate Matter – PM1, PM2.5, PM4, and PM10

Particulate matter is solid or liquid material suspended in the air, measured in micrograms per cubic meter ($\mu g/m^3$). The number after "PM" refers to the size of particle being counted, in micrometers.

- **PM1** – the smallest particles the sensor tracks, roughly the size of smoke and combustion byproducts.
- **PM2.5** – particles 2.5 micrometers and smaller. This is the single most important particulate number, and the one most government air quality indexes are built around. Particles this small bypass the body's natural filtering in the nose and throat and reach deep into the lungs, and can pass into the bloodstream. Wildfire smoke, vehicle exhaust, and cooking smoke are all dominated by PM2.5-sized particles.
- **PM4** – a midpoint size class, less commonly referenced outside of raw sensor data, but useful as a transition point between fine and coarse particulates.
- **PM10** – particles up to 10 micrometers, which includes dust, pollen, and mold spores. These are still inhalable but are mostly filtered out before reaching deep into the lungs, so PM10 matters more for respiratory irritation than the deeper health risks associated with PM2.5.

The generally accepted safe range for PM2.5 is under 12 $\mu g/m^3$. Between 12 and 35 is considered elevated. Above 35 is considered unhealthy, particularly for sensitive groups. These are the exact thresholds the dashboard uses to color PM2.5 readings green, yellow, or red, both on the physical screen and on the web dashboard's charts.

VOC Index

VOC stands for volatile organic compounds, a broad category covering off-gassing from things like cleaning products, paint, new furniture, and adhesives. The SEN54 does not report a raw concentration for any single compound. Instead, it runs an onboard algorithm that establishes a baseline for the air it has been breathing and reports an index, typically centered around 100, where higher numbers mean the air smells more different than usual to the sensor, not that a specific chemical has crossed a specific danger line. Because there is no single, widely agreed safe/unsafe threshold for this index the way there is for CO2 and PM2.5, the dashboard displays it for awareness but does not use it to drive the overall air quality status.

How the Status Indicator Is Computed

Both the physical screen and the web dashboard show a single overall air quality verdict, GOOD, ELEVATED, or POOR, so that a glance at either one answers the only question that usually matters: is something worth doing about the air in this room right now. That verdict is computed from the worse of the CO2 and PM2.5 bands described above. VOC index is left out for the reason given in its own section; temperature and humidity are left out because they are comfort metrics rather than air quality readings.

Software Architecture

Firmware development was assisted by Claude Code.

Three-Layer Design

The code is organized into three separate jobs that never overlap.

Layer	Responsibility
Data Layer	Fetches data from the network or hardware sensors and saves the results into shared memory. Never touches the display.
Display Layer	Reads that shared memory and draws to the screen. Never makes network calls or reads hardware directly.
Coordinator	The device's main loop – the code that runs over and over, forever. Owns all timing decisions, only makes network calls when WiFi is actually connected, and decides when each layer runs.

This separation means that adding a new sensor is just filling in two functions. One reads the hardware and saves the result into shared memory, and the other reads that memory and draws it to the screen. Nothing else in the codebase needs to change.

Struct-Scoped State

Earlier versions of this project kept every piece of live data as its own separate, loosely-organized variable. As the project grew, this made it hard to tell at a glance which pieces of data were related, or which part of the code was really responsible for each one. Related data, sensor readings, touch input, saved settings, and menu state, is now grouped into a small number of purpose-built bundles instead, which makes the code considerably easier to navigate as the project keeps growing.

Non-Blocking Loop

The device's main loop never pauses or waits. Every repeating task checks the clock and only runs once its own interval has passed, rather than using a command that would freeze the whole program for a set amount of time. This keeps the loop running continuously, so touch input always feels instant and no task can hold up another.

```
// All timing decisions live in loop() -- nothing blocks
if (currentMillis - lastClockTime >= clockInterval) {
    checkWiFiHealth();
    if (!menuOpen) updateTimeDisplay();
    checkDaylightSavings();
    lastClockTime = currentMillis;
}
```

Canvas Buffering

Several regions of the screen are drawn into memory first, rather than straight onto the display, which prevents flickering on every part of the screen that updates on its own schedule. Without this, each small drawing step would be sent to the screen separately, and the eye would catch the screen passing through partially-drawn states as a flash. With this approach, an entire region is finished in memory first and then sent to the display in one clean update, so nothing ever appears half-drawn.

Color Palette System

All the colors used on screen are defined in one place, a single color scheme that the rest of the code reads from. Night Mode was extended in v2.0 to cover the settings menu, brightness slider, and reset confirmation dialog in addition to the main dashboard, so the entire interface now shifts to the red color scheme together, rather than leaving the menu at full brightness while the dashboard dims around it. Adding a new display element still just requires reading from that shared color scheme rather than hardcoding a color by hand.

Settings Menu

Tapping anywhere on the dashboard opens a scrollable touch-driven settings menu. Current settings include Night Mode, temperature unit (F/C), military time (12/24hr), brightness, Auto Dim, and WiFi reset, with scroll arrows to page through the list and a live scrollbar showing the current position. Every tappable button on screen, the close button, the scroll arrows, and the reset confirmation's YES/NO buttons, shares its exact position between the code that draws it and the code that checks for a tap, so a button can never silently drift away from where it actually responds to a touch.

Settings Persistence

All user settings are saved to memory that survives a power loss the moment they are changed, and restored the next time the device starts up, before anything is drawn to the

screen. This means the device always starts up exactly as the user left it. A deep reset clears both the saved settings and the WiFi credentials.

The Web Dashboard

Motivation

The physical display has always been, by design, a snapshot of right now. Adding a scrolling graph to a 480x320 panel that already shows a clock, weather, and nine sensor readings was never going to work without crowding everything else off the screen. Rather than compromise the physical dashboard, v2.0 adds a second, independent way to look at the same data: a web page served directly from the device itself, reachable at `http://omnicore.local` from any browser on the same network, with as much screen space as a laptop or phone can offer.

Live Data Endpoint

The device serves a small web address, `/data`, that returns the current value of every live sensor reading in a standard, machine-readable format (JSON). The web page checks in with this address every two seconds and updates a grid of live cards, one per reading, each colored the same green, yellow, or red the physical screen would use for the readings that have defined safety thresholds.

History Ring Buffer

Alongside the live readings, the device keeps a 24-hour rolling history entirely in working memory (RAM). One sample is taken every 60 seconds and stored in a fixed-size list of 1,440 slots that wraps around on itself: once the list is full, each new sample simply overwrites the oldest one automatically, rather than the list ever growing. There is no separate "delete anything older than 24 hours" logic anywhere in the code; the 24-hour window falls directly out of the list's size (1,440 slots) multiplied by the sample interval (60 seconds), which comes out to exactly 24 hours.

This history is intentionally kept in working memory only, not written to permanent storage. The device runs continuously, so losing history on the rare reboot for a firmware update was judged an acceptable tradeoff against the real cost of writing to permanent storage every minute for months, which would meaningfully wear it down over the device's lifetime.

A separate web address, `/history`, serves the entire list of readings, oldest point first, every time it is requested. Every history sample is timestamped using the device's internet-synchronized clock, rather than its own internal uptime counter, so that the browser and the device always agree on what time a given point actually happened. An earlier version of this feature used timestamps based on how long the device had been running, which caused the browser's live-appended points and the device's historical points to disagree about what time it was, producing a visibly broken, jumping timeline. Switching both sides to the same real-world clock resolved it completely.

Charts and Range Selection

The web dashboard renders five charts: CO2, particulate matter (all four PM size classes on one chart), VOC index, temperature, and humidity. The CO2 and PM2.5 lines change color along their length using the same thresholds as the live cards, so a line visibly shifts from green to yellow to red as a room's air quality changes over the day, rather than requiring a viewer to read the numbers to notice a problem.

A four-way range toggle (1 hour, 6 hours, 12 hours, 24 hours) lets a viewer zoom into recent fine detail or zoom out to the full day. The device is never asked for a different amount of data depending on which range is selected; the full 24-hour history is fetched once every 60 seconds and kept in the browser's own memory, and switching ranges simply trims that already-downloaded data down to the most recent portion, which is instant and does not need another trip to the device.

Figure 12: Web dashboard – live cards and the CO2 chart.

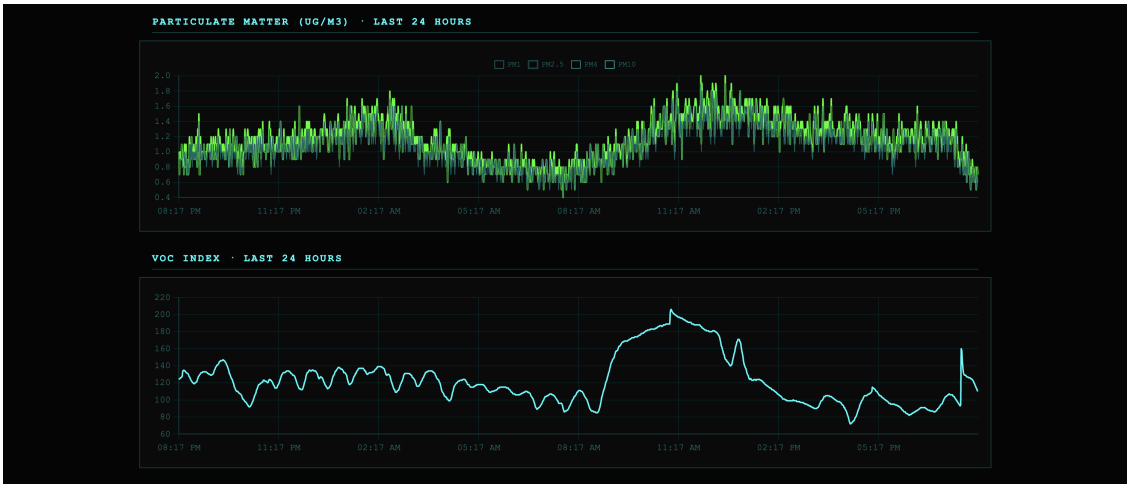

Figure 13: Web dashboard – particulate matter and VOC index charts.

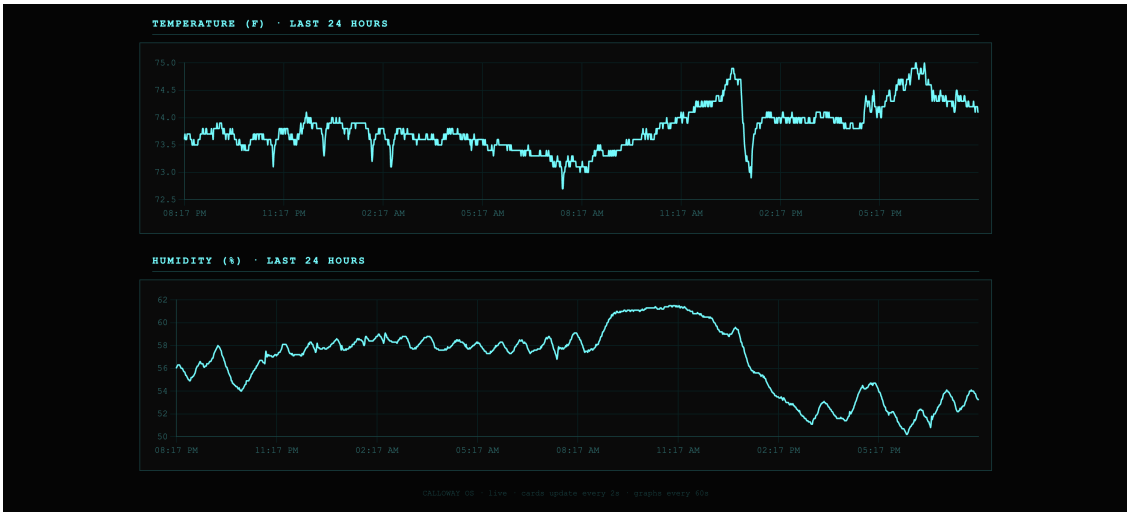

Figure 14: Web dashboard – temperature and humidity charts.

Air Quality Status Indicator

Both the web dashboard and the physical screen now display the same overall air quality verdict, computed identically in both places from the worse of the current CO2 and PM2.5 bands, as described in Section 4. Keeping this logic conceptually identical in both places, even though it is implemented once in C++ for the screen and once in JavaScript for the browser, was a deliberate choice, so that the two views of the device can never silently disagree about whether the room's air is fine or not.

A Note on Multi-File Builds

An attempt was made during this phase of development to split the firmware's single source file into several smaller files for readability, a common practice in larger software projects. This was reverted. The specific combination of build tools this project uses, PlatformIO (the build system), the pioarduino platform (its ESP32 support layer), and this project's pinned version of arduino-esp32 (the underlying ESP32 framework), produces an intermittent error whenever more than one of those files references the WiFi networking code, striking a different file on nearly every rebuild. After ruling out every explanation in the actual code, this was concluded to be an instability in that specific build-tool combination rather than a problem with the code; the original single-file version has never failed under the same conditions. The one piece of that experiment kept is a separate file holding the web dashboard's HTML, CSS, and JavaScript, the code that builds and styles the web page, which gets pulled back into the main file automatically when the firmware is built. That kind of file does not create the same problem a fully separate source file does, so this keeps the readability benefit without triggering the build issue.

Reliability Design

Reliability was treated as a first-class requirement, not an afterthought. The following mechanisms work together to ensure the device recovers from any failure without manual intervention.

Mechanism	What it handles
Safety timer (30s)	Resets the chip if the main loop ever stalls, catching stuck network requests, driver issues, or any other freeze
Network timeouts (5s response, 3s connect)	Prevents a slow or unresponsive online service from freezing the display
WiFi auto-reconnect (60s retry)	Quietly retries the connection in the background without disrupting the display
Time resync limit	Only resyncs the clock if it has been more than 60 seconds since the last sync, preventing repeated attempts during a spotty connection
6-hour restart	If WiFi never comes back after 6 hours, the device restarts cleanly rather than sitting in a broken state
Daylight saving detection	Resyncs location and time at 2:01 AM once per year to handle the twice-yearly time change automatically

The safety timer is intentionally turned on only after every startup step that could take a while has finished, to prevent false alarms during the WiFi connection process on boot, which can take an unpredictable amount of time depending on network conditions. During this phase of development it was discovered that recent versions of the arduino-esp32 framework (the underlying ESP32 software layer this project builds on) automatically turn on their own version of this same safety timer before the device's own startup code ever runs, which silently caused this device's own 30-second timeout and restart settings to never actually take effect, since starting a second timer on top of one that is already running is rejected. The fix explicitly turns off whatever safety timer is already running immediately before applying this device's own settings, guaranteeing the intended configuration is the one actually in effect, rather than silently losing out to whatever the framework set up first.

Development History

The project was developed iteratively, with each version focused on a single clear goal.

v1.0 – Core Functionality

Established the foundation of Calloway OS. Implemented NTP-synchronized timekeeping and weather data from OpenWeatherMap. This version proved that the core concept, a

glanceable dashboard driven entirely by internet data, was viable on the ESP32-C6 platform.

v1.1 – Network Intelligence

Replaced WiFi credentials that were built directly into the code with a temporary setup network the device creates on first boot, letting the network name and password be entered through a webpage instead of reflashing the firmware. Added automatic location detection via ip-api.com (their free tier is HTTP-only), which allowed timezone and weather city to be set dynamically from the network. The device became truly self-configuring.

v1.2 – Architecture Overhaul

Restructured the entire codebase around the three-layer design described in Section 5. Introduced the non-pausing timing approach also described there, separated data-fetching and display-drawing code, and moved all timing decisions into the main loop.

v1.3 – Reliability

Added the hardware watchdog, HTTP timeouts, WiFi health monitoring with auto-reconnect, the deep reset system, and the 6-hour restart fallback.

v1.4 – Hardware and Display

Brought the TSL2591 light sensor online. Added a TFT (thin-film transistor) display using the memory-buffered drawing approach described in Section 5 to prevent flickering. Implemented the Night Mode color scheme and the vertical light-level bar on the display.

v1.5 – Settings and User Control

Added a scrollable settings menu with Night Mode, Celsius/Fahrenheit toggle, Military Time, Brightness control, Auto Dim, and WiFi reset with confirmation. All settings persist across reboots using the device's built-in non-volatile storage (memory that survives a power loss).

v1.6 – Local Sensor Integration

Brought a local temperature and humidity sensor online via the BME280, displayed in the top left of the dashboard. Added canvas buffering for the local sensor area and the date line, eliminating the last remaining sources of display flicker. A counterfeit SGP30 air quality sensor was also identified and discarded during this version, its serial ID memory was blank and its output values were impossible; a genuine replacement was ordered but a different sensor path was ultimately taken in v1.7. This LaTeX report itself was also started during this version.

v1.7 – Real CO2 and VOC

Added the SCD40 for genuine CO2, temperature, and humidity, and the SGP41 for VOC and NOx (nitrogen oxide gases). This version also found and fixed a real reliability issue on the shared sensor connection: the TSL2591 was leaving that connection in a bad state that broke communication with the SCD40 when powering on from fully off, fixed by explicitly resetting the connection between starting up the two sensors.

v1.8 – Larger Display

Upgraded from the 2.8" ILI9341 to the 3.5" ST7796 at 480x320, switching to the Arduino_GFX library in the process. The larger, higher-resolution screen is what made room for everything added in the versions since.

v1.9 – Touch Input

Replaced the hardware button entirely with an FT6236 capacitive touch controller. The settings menu was reworked around touch: scroll arrows, a brightness slider with live drag, and a two-step YES/NO confirmation before a WiFi reset is allowed to happen. The WiFi connection screen gained a hold-to-reset gesture as an escape hatch. The backlight also moved to direct PWM control, since the new display has its own built-in driver circuitry and no longer needs an external transistor to switch its power. Auto Dim, mapping ambient light to brightness, was implemented here. Getting touch input reliable took some real debugging: a polling rate cap, a minimum tap duration, and a cooldown between taps were all needed to stop the touch controller from registering phantom touches.

v2.0 – Particulate Matter and the Web Dashboard

The SGP41 was replaced by the SEN54, adding real particulate matter readings (PM1, PM2.5, PM4, PM10) alongside its own VOC index. PM2.5 is now color-coded using the same thresholds, in the same style used by the U.S. Environmental Protection Agency (EPA), described in Section 4. This version also added the web dashboard: a live data address serving up-to-the-second readings, a 24-hour in-memory history, and a browser-based set of charts with a 1/6/12/24-hour range toggle, described in full in Section 6.

Current Status and Future Work

What's Working Now

The device currently displays a live NTP-synchronized clock with 12 or 24-hour format, the current date, internet weather data including temperature, daily high and low, and sky conditions, local CO2, temperature, and humidity from the SCD40, full particulate matter and VOC readings from the SEN54, a live ambient light bar from the TSL2591, an overall air quality status indicator, a WiFi signal strength indicator, and a full touch-driven settings menu. All screen regions update without flickering thanks to the memory-buffered drawing approach used throughout. Separately, a live web dashboard at `omnicore.local` mirrors the

same data with a 24-hour history and interactive charts, viewable from any device on the same network. The custom PCB and 3D printed enclosure described in Sections 3.6 and 3.7 are both built and in use.

Next Steps

- **Matter integration.** Investigating whether the SEN54's particulate readings can be exposed through Matter, a smart-home connectivity standard, pending either upstream support from Espressif's Arduino core or a hand-written wrapper following the pattern used by existing community air-quality Matter implementations.
- **Sleep timeout.** An additional menu setting to dim or turn off the display after a configurable period of inactivity.

Conclusion

The Omni-Core Telemetry Station started as a desk clock and grew into something more interesting. It is a small device that knows where it is, keeps itself alive, and shows useful real-world data without being told what to do, and it now does that in two places at once: a glanceable physical screen for right now, and a web dashboard for everything that led up to it. The engineering philosophy that shaped it, thinking about failure modes before writing code, keeping functions honest about what they do, and making the architecture easy to extend, is the same philosophy that shows up in professional embedded systems at every scale, and the same philosophy that made it possible to add an entire second interface to this device without touching the reliability guarantees the first one was built on.

The hardware is stable, all sensors and touch input are fully online, and the device now runs on a custom PCB inside its own 3D printed enclosure. Two real bugs (a safety timer that was silently never actually running, and a history timeline that drifted out of sync over long sessions) were found and fixed along the way. What remains is mostly software: Matter support and a sleep timeout, both described in Section 9.2.

Appendix: Pinout and Wiring Reference

This appendix collects the complete electrical reference for the build: every pin assignment on the XIAO ESP32-C6, the shared I2C bus and device addresses, and the supporting power/decoupling network, as built on the current PCB revision described in Section 3.6.

XIAO ESP32-C6 Pin Assignments

Pin	Net	Function
D0	LCD_RST	Display reset
D1	LCD_CS	Display chip select
D2	PWM	Backlight, direct PWM drive to display LED pin
D3	LCD_RS	Display data/command (DC)
D4	SDA	I2C data – shared by all sensors and touch
D5	SCL	I2C clock – shared by all sensors and touch
D6	–	Unused
D7	CTP_RST	Touch controller hardware reset
D8	SCK	Display SPI clock
D9	CTP_INT	Touch controller interrupt (wired, not currently used in firmware)
D10	MOSI	Display SPI data

The display is write-only over SPI; no MISO connection is present.

Shared I2C Bus

All I2C devices, the touch controller and all three environmental sensors, share a single bus on D4 (SDA) and D5 (SCL), running at 400kHz and pulled up to 3.3V with 4.7kΩ resistors on both lines.

Device	Function	I2C Address
FT6236	Capacitive touch controller	0x38
SCD40	CO2 / temperature / humidity	0x62
SEN54	Particulate matter (PM1/2.5/4/10) + VOC index	0x69
TSL2591	Ambient light	0x29

Power and Decoupling

- Bulk 10µF capacitors sit on both main supply rails, one on 3.3V and one on 5V.
- 1µF capacitors decouple the SCD40 and the TSL2591 individually.
- A 10µF bulk capacitor decouples the SEN54's supply pins, sized larger than the other sensors to cover its internal fan's current draw.
- 100nF ceramic capacitors are placed at the SCD40, TSL2591, display FPC connector, and SEN54 supply pins, as close as possible to each device's power pin, to keep

high-frequency switching noise from coupling into the shared I2C bus.

- I2C pull-ups: 4.7kΩ on both SDA and SCL to 3.3V.

Datasheet Links

Component	Source
XIAO ESP32-C6	Seeed Studio Wiki
ST7796 3.5” display module	LCD Wiki (manufacturer)
FT6236 touch controller	Adafruit
TSL2591 light sensor	ams (via Adafruit)
SCD40 CO2 sensor	Sensirion
SEN54 particulate sensor	Sensirion

Bill of Materials

Designator	Component	Part Number
U1	XIAO ESP32-C6	1597-113991254-ND
U2	SCD40-D-R2	1649-SCD40-D-R2CT-ND
U4	TSL25911FN	TSL25911FNCT-ND
CN1	SM06B-GHS-TB (SEN54 header)	455-1568-1-ND
FPC	FFC2B35-14-G (display connector)	2073-FFC2B35-14-GCT-ND
C2, C4	10µF 0402 cap	490-GRM155R60J106ME05DCT-ND
C5, C7	1µF 0402 cap	1276-1067-1-ND
C1, C3, C6, C8-C10	100nF 0402 cap	1276-CL05B104KP5VPNCCT-ND
R1, R2	4.7kΩ 0402 resistor	311-4.7KJRCT-ND
R3	100Ω 0402 resistor	541-3239-1-ND
–	SEN54 (particulate/VOC sensor)	SEN54-SDN-T

Figures

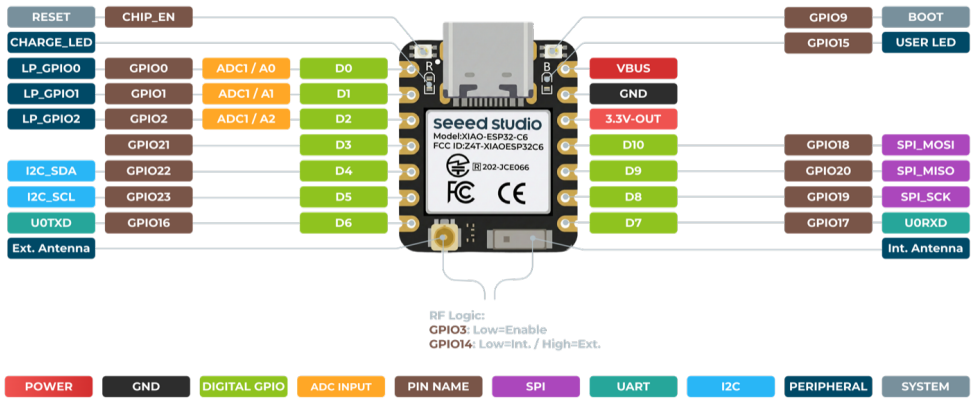

Figure 15: XIAO ESP32-C6 pinout, front (Seeed Studio).